



TP 3

IPC

Programación sobre redes
Instituto Ind. Luis A. Huergo

Contents

1	Introducción	2
1.1	La misión continúa	2
1.2	Qué hay a bordo	2
1.3	Tierra	2
1.4	La caja negra	3
1.5	Tormentas solares	3
1.6	Objetivo	3
2	Consigna	4
2.1	Arquitectura	4
2.2	El código inicial	5
2.3	Reglas	5
2.4	El contrato	5
2.4.1	La carpeta de la caja negra	5
2.4.2	Señales	6
2.4.3	Códigos de salida y apagado	7
2.5	Parte 0: Las herramientas	7
2.6	Parte 1: A bordo	7
2.6.1	El subsistema	7
2.6.2	La nave	7
2.7	Parte 2: La caja negra	8
2.8	Parte 3: La tormenta solar	9
2.9	Informe	9
2.10	Librerías externas	9
3	Rubricas	10
3.1	Rúbricas	10
3.2	Posible defensa oral	10
4	Entrega	12
4.1	Grupos	12
4.2	Método de entrega	12
4.3	Fecha de entrega	12

1

Introducción

1.1 La misión continúa

En el primer trabajo práctico, el [Instituto de Investigación y Lanzamiento Aeroespacial Huergo \(I.I.L.A.H.\)](#) puso en marcha su programa de exploración espacial y lanzó sus naves desde distintas ciudades. Una de ellas, la **AO-6X**, despegó de Puerto Kernel con destino a **Nebulix**. Hoy está en pleno vuelo, y el Instituto necesita lo que todavía le falta: la **computadora de a bordo**.

En este trabajo van a construir esa computadora. No la nave, que ya está lanzada, ni la física del vuelo, que ya viene simulada: el software que coordina, a bordo, un conjunto de módulos independientes, y que mantiene comunicada a la nave con Tierra.

1.2 Qué hay a bordo

La nave tiene cuatro subsistemas: **reactor**, **soporte vital**, **navegación** y **comunicaciones**. Cada uno corre en su propio módulo de cómputo (para nosotros, en su propio **proceso**), reporta telemetría periódicamente y acepta órdenes. Los subsistemas se desgastan con el tiempo; se pueden reparar; y, de vez en cuando y sin aviso, se cuelgan. Un subsistema colgado no reporta nada, no responde a nada, y lo único que se puede hacer es reemplazarlo por uno nuevo.

La computadora de a bordo es la que lanza los subsistemas, los escucha, les da órdenes, se da cuenta cuando uno se cae y lo reemplaza. Y la que deja todo registrado en la caja negra.

1.3 Tierra

Control de Misión sigue en Puerto Kernel. A esta altura del viaje la nave está a varios minutos luz, así que no hay diálogo en tiempo real: Tierra manda **órdenes discretas**

y, cada tanto, se sincroniza la caja negra. Control de Misión puede conectarse y desconectarse cuando quiera, y puede haber varios operadores mirando la nave al mismo tiempo.

1.4 La caja negra

Todo lo que pasa a bordo queda registrado en la **caja negra**: el estado de cada subsistema, cada caída, cada reinicio y cada orden. Es lo único que sobrevive a cualquier cosa, incluso a que la computadora de a bordo se reinicie en pleno vuelo, y es lo que Tierra lee.

1.5 Tormentas solares

El viaje a Nebulix atraviesa una región de tormentas solares. Los efectos son variados: subsistemas que se cuelgan, ráfagas de órdenes desde Tierra y, cada tanto, un reinicio completo de la computadora de a bordo. La cátedra entrega un programa, caos, que simula esas tormentas. La nave tiene que seguir volando igual.

1.6 Objetivo

El objetivo del trabajo es que apliquen los mecanismos de comunicación entre procesos que vimos en clase y en el taller: **pipes**, **señales** y **archivos compartidos**. Cada uno tiene en este trabajo un rol propio, y parte del trabajo es entender por qué ese rol y no otro.

- **Pipes**: un flujo ordenado de bytes entre padre e hijos. Acá, la telemetría y las órdenes entre la nave y sus subsistemas.
- **Señales**: un aviso sin datos, entre cualquier par de procesos. Acá, “pasó algo”: apagar, un hijo que murió, hay órdenes nuevas.
- **Archivos compartidos**: estado que persiste y que cualquiera puede leer por nombre. Acá, la caja negra, y el enlace con Tierra.

Al terminar van a tener un sistema de varios procesos al que le pueden matar cualquier parte, incluso el coordinador, y que sigue funcionando. Ese es, en el fondo, el problema central de la programación sobre redes: coordinar programas que fallan por separado.

2

Consigna

2.1 Arquitectura

El sistema está formado por cuatro programas. Ustedes escriben dos: la nave y el subsistema.

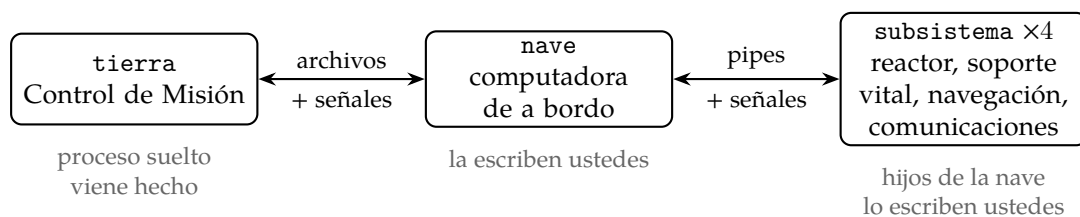


Figure 2.1: Los procesos y cómo se comunican

- nave es la **computadora de a bordo**. Lanza un proceso subsistema por cada uno de los cuatro roles, conversa con cada uno por un par de pipes y mantiene la caja negra. Es el proceso coordinador y donde va la mayor parte del trabajo.
- subsistema es **un subsistema**. Habla únicamente por su entrada y salida estándar: recibe órdenes por stdin y emite telemetría por stdout. No sabe si del otro lado hay una terminal o la nave, y eso es a propósito: se puede probar a mano antes de enchufarlo.
- tierra es **Control de Misión**, y **viene hecho**. Es un proceso suelto: no es hijo de la nave ni la nave es hija de él, y lo único que comparte con ella es la carpeta de la caja negra. Léanlo: es el ejemplo de cómo se habla con un proceso que no es pariente.
- caos es la **tormenta solar**. Mata subsistemas, manda ráfagas de órdenes y, cada tanto, mata la nave con `kill -9`. Viene hecho.

2.2 El código inicial

- `nave.cpp`: la estructura ya está armada (la configuración, lo que la nave sabe de cada subsistema, los handlers de señales, el tablero, la bitácora, el snapshot y el loop principal). Faltan seis métodos, marcados con `TODO`, que es donde está el IPC.
- `subsistema_main.cpp`: el parseo de argumentos y la creación del `Subsystem`. Falta el loop principal.
- `tierra.cpp`: hecho.
- `ipc/`: los helpers de IPC. Cinco funciones están **vacías y las completan ustedes** en la parte 0; el resto viene hecho.
- `sim/`: la clase `Subsystem`, que simula el desgaste, las reparaciones y las fallas de un subsistema. **No la modifiquen**: los tests la verifican tal cual está, y no sabe nada de procesos ni de pipes.
- `ejemplo/`: `lanzar.cpp` y `hijo.cpp`, el patrón `fork + exec + dup2` para lanzar otro programa y hablarle por pipes, que vimos en clase. Anda desde el primer día.
- `tests/`: los tests de `ctest` prueban `ipc/` y `Subsystem`; `integracion.py` levanta la nave de verdad y verifica el contrato desde afuera. **No los modifiquen**. Van a fallar hasta que implementen cada parte: úsenlos como guía.
- `caos/`: `caos.py` es la tormenta solar, y `verificar_caja_negra.py` revisa que una caja negra cumpla el contrato; con `--vigilar` detecta snapshots rotos.

2.3 Reglas

- **Solo procesos**. Nada de `thread`, `mutex` ni `semaphore`.
- **Solo pipes anónimos, señales y archivos**. Nada de FIFOs, sockets, colas de mensajes ni memoria compartida.
- **Nada de `select()`, `poll()`, `epoll` ni `kqueue`**. Todavía no los vimos. Con `read()` bloqueante o no bloqueante, señales y `usleep()` alcanza.
- Un subsistema habla **solo** por `stdin` y `stdout`. Los logs de debug van por `stderr`, que nadie captura.
- `sleep` y `usleep` sirven para simular tiempo o para no quemar CPU en un loop, **no para sincronizar**.
- Tiene que compilar y correr en Linux, con el `CMakeLists.txt` que se entrega.

2.4 El contrato

2.4.1 La carpeta de la caja negra

Por defecto es `caja_negra/` en el directorio actual; se cambia con `--caja_negra <dir>` en los tres programas. La nave la crea si no existe.

Una línea `subsistema=` por cada rol, con: rol, estado (OK, ALERTA, CRITICO o CAIDO),

Archivo	Quién escribe	Qué es
nave.pid	la nave	su PID, una línea. Se crea al arrancar y se borra al apagarse bien.
estado.txt	la nave	el último snapshot del estado, escrito de forma atómica, al menos una vez por segundo.
bitacora.log	la nave	todo lo que pasa, una línea por evento, con timestamp. Solo se agrega.
ordenes.txt	tierra	las órdenes, una por línea, numeradas. Solo se agrega.

Table 2.1: Los archivos de la caja negra

Listing 1: Formato de estado.txt

```

1 ultima_orden=7
2 subsistema=reactor OK 71.3 4242
3 subsistema=soporte_vital ALERTA 28.0 4243
4 subsistema=navegacion CAIDO - -
5 subsistema=comunicaciones CRITICO 8.1 4245
6 FIN

```

nivel y PID. Para un subsistema caído, los dos últimos campos llevan un guion. La última línea es **siempre** FIN: un lector que no la encuentra sabe que el archivo está incompleto.

Listing 2: Formato de ordenes.txt

```

1 1 REPARAR todos
2 2 REPARAR reactor
3 3 ABORTAR

```

El primer campo es el número de secuencia, empieza en 1 y crece de a uno. Las órdenes son dos: REPARAR, con un rol o todos, que la nave le reenvía al subsistema por su pipe y le devuelve el nivel a 100; y ABORTAR, el apagado ordenado. La nave guarda en el snapshot el número de la última orden procesada y, cuando le avisan, procesa **todas** las que tengan un número mayor. Nunca una sola.

2.4.2 Señales

Señal	A quién	Qué significa
SIGTERM, SIGINT	nave	apagado ordenado
SIGUSR1	nave	hay órdenes nuevas en ordenes.txt
SIGCHLD	nave	la manda el kernel: un hijo terminó
SIGTERM	subsistema	terminar el tick en curso y salir con 0

Table 2.2: Las señales que tienen significado fijo

2.4.3 Códigos de salida y apagado

Un subsistema termina con 0 si le pidieron salir (SIGTERM o fin de la entrada) y con 2 si falló: cuando `failed()` da `true`, el proceso termina **en el acto**, sin despedirse. Es un *crash*, y la nave tiene que enterarse sola.

Apagar la nave, sea por señal o por la orden ABORTAR, es: mandar SIGTERM a cada subsistema vivo, esperar hasta dos segundos a que terminen, SIGKILL a los que sigan, `waitpid` de todos, un último snapshot y una línea en la bitácora, borrar `nave.pid` y salir con 0. Después de eso no puede quedar ningún proceso vivo ni ningún zombie.

2.5 Parte 0: Las herramientas

En `ipc/` hay cinco funciones vacías. Son las que van a llamar decenas de veces en las partes que siguen, así que son lo primero. Cada una tiene arriba el contrato exacto que tiene que cumplir, y los tests de `ctest` lo verifican.

- `write_all`: `write()` puede escribir de menos. Hay que insistir.
- `read_line`: `read()` no devuelve líneas. Hay que armarlas, y guardar lo que sobra.
- `set_nonblocking`: el `fcntl()` bien hecho, sin pisar los otros flags.
- `write_atomic`: un temporal y un `rename()`, para que nadie lea un archivo a medias.
- `install_handler`: `sigaction()`, y la decisión sobre SA_RESTART.

Si hicieron el taller, ya escribieron las tres primeras. Tráiganlas.

2.6 Parte 1: A bordo

Es la parte de los **pipes**, y de las señales que hacen falta para tener a los subsistemas bajo control. Arranca en el ejercicio 4 del taller: la mesa de operadores es la nave, los sensores son los subsistemas. No lo rehagan, tráiganlo.

2.6.1 El subsistema

Hace `tick()` cada periodo milisegundos y emite por `stdout` una línea de telemetría. Mientras tanto **tiene que poder recibir una orden por `stdin`** sin dejar de emitir; hay más de una forma correcta de resolver eso, y la de ustedes va explicada en el informe. Si `failed()` da `true`, termina en el acto con código 2. Si se le cierra la entrada, o le llega SIGTERM, termina con 0. El formato de las líneas es de ustedes.

2.6.2 La nave

Los seis métodos `TODO` de `nave.cpp`, en este orden:

- `launch`: lanzar un subsistema con `fork + exec + dup2`, como en el ejemplo, con el extremo de lectura en modo no bloqueante.
- `read_telemetry` y `process_line`: leer lo que haya en los cuatro pipes sin quedarse trabada en ninguno, y entender lo que manda su subsistema. Un EOF en el pipe es un subsistema que murió.
- `reap_children`: cuando llega `SIGCHLD`, `waitpid` de todos los que hayan terminado, anotar en la bitácora cómo murió cada uno, y programar el relanzamiento. El loop ya relanza solo.
- `shutdown`: el apagado ordenado del contrato. **Ctrl+C en la terminal de la nave tiene que dar exactamente el mismo resultado que `kill -TERM`.** La primera vez no va a pasar; entender por qué es la pregunta 2.

Con eso el tablero, que ya viene hecho, se imprime cada segundo:

Listing 3: El tablero

```

1 [ 47.20s] [nave    ] --- tablero ---
2 [ 47.20s] [nave    ] reactor      OK      71.3  hace 0.3s
3 [ 47.20s] [nave    ] soporte_vital ALERTA  28.0  hace 0.1s
4 [ 47.20s] [nave    ] navegacion   CAIDO    -      murió con exit(2)
5 [ 47.20s] [nave    ] comunicaciones CRITICO  8.1  hace 0.4s

```

Preguntas:

1. Maten la nave con `kill -9`. ¿Qué ve cada subsistema en su `stdin`? ¿Qué le pasa cuando intenta escribir la siguiente telemetría? ¿Quién es su padre ahora? Y al revés: ¿qué ve la nave en el pipe cuando muere un subsistema?
2. Ctrl+C en la terminal de la nave: ¿a quién le llega `SIGINT`? ¿Qué pasó la primera vez que lo probaron, y qué cambiaron para que el apagado fuera ordenado?

2.7 Parte 2: La caja negra

Es la parte de los **archivos**, y donde entra Tierra. La nave ya escribe `nave.pid` y el snapshot en el loop principal; lo que falta es el único método `TODO` que queda:

- `process_orders`: cuando llega `SIGUSR1`, leer `ordenes.txt` y procesar todas las órdenes que todavía no se procesaron. `REPARAR` va al subsistema por su pipe; `ABORTAR` apaga.

Prueben con tierra: en otra terminal, `./tierra` ver muestra el estado, y `./tierra ordenar REPARAR` todos deja la orden y le avisa a la nave.

Preguntas:

1. Escriban el snapshot con un ofstream común en vez de `write_atomic`, y corran `verificar_caja_negra.py --vigilar 10` mientras la nave anda. ¿Qué reporta? ¿Por qué `rename()` arregla eso?
2. ¿Por qué las órdenes no viajan en la señal? Manden dos órdenes en el mismo instante (`./tierra ordenar REPARAR reactor & ./tierra ordenar REPARAR navegacion`). ¿Cuántos SIGUSR1 le llegan a la nave? ¿Cuántas órdenes procesa? ¿Qué pasaría si procesara “una orden por señal”?

2.8 Parte 3: La tormenta solar

caos corre contra su nave durante unos minutos, la castiga y produce un reporte: cuántos reinicios hubo, si la caja negra quedó consistente y qué falla la rompió, si algo la rompió. Corran caos al menos tres veces, adjunten los reportes en el repositorio y arreglen lo que rompió.

2.9 Informe

Corto, de dos a tres páginas, y con esto:

- Una tabla con cada mecanismo, para qué lo usaron y **por qué no otro**. Es la parte más importante del informe.
- El protocolo entre nave y subsistema: qué líneas van en cada sentido.
- Para cada proceso, dónde bloquea y dónde no, y por qué.
- Qué rompió caos y qué cambiaron.
- Las respuestas a las preguntas.

Se evalúa la claridad, no la extensión.

2.10 Librerías externas

El uso de librerías externas está permitido bajo los mismos criterios de siempre:

- No pueden resolver una porción significativa de lo que se está evaluando, sí partes superfluas del trabajo.
- Se debe justificar el uso de cada librería y su impacto en la solución propuesta.
- No deben requerir instalaciones externas o configuraciones especiales para compilar y ejecutar el proyecto.

Ante cualquier duda sobre si una librería entra en estas reglas, consúltenlo antes de usarla.

3

Rúbricas y criterios de corrección

3.1 Rúbricas

Los criterios de corrección a utilizar serán los siguientes:

- **Tests:** los tests de ctest y los de integración son el principal criterio de corrección. Dependiendo de cuántos y cuáles se ejecuten correctamente, la nota (y eventualmente la aprobación) variará drásticamente. Sin embargo, la correcta ejecución de la totalidad de los tests no garantiza de manera unilateral la aprobación del trabajo práctico, ni la máxima nota.
- **Adecuación a la consigna:** el contrato de la sección 2.4 se cumple tal cual, el sistema compila y corre en Linux como está documentado, y después de un apagado ordenado no queda ningún proceso vivo ni zombie. Se prueba matando cosas, y con caos.
- **Decisiones y criterios de diseño:** cada mecanismo está donde tiene que estar y ustedes saben por qué. Dónde bloquea cada proceso, cómo se enteran de cada evento, qué pasa cuando algo muere. Se evalúa tanto en el código como en el informe, y vale más una decisión justificada que una decisión perfecta sin justificar.
- **Prolijidad, legibilidad y buenas prácticas:** en menor medida, la prolijidad y legibilidad del código afectará a la nota. Cerrar los descriptores que no se usan, esperar a los hijos, no hacer trabajo adentro de un handler: acá eso no es estilo, es corrección.

3.2 Posible defensa oral

Utilizando los criterios de corrección mencionados en la sección anterior, el docente concluirá una nota tentativa. Esta nota (así como el mismo estado de aprobación) está

sujeta y podrá verse modificada por una posible defensa oral a realizarse el día **lunes 28 de septiembre**.

4

Información sobre la entrega

4.1 Grupos

El trabajo se resuelve en grupos de mínimo 2 y máximo 3 integrantes. **Sin excepciones.** Recomendamos fuertemente que **no se dividan** las tareas: el trabajo está pensado para resolverse haciendo *pair programming*, y en este en particular la nave y el subsistema tienen que ponerse de acuerdo en demasiadas cosas como para escribirlos por separado.

4.2 Método de entrega

Igual que en los trabajos anteriores: el repositorio del grupo en GitHub Classroom es la entrega. Hagan **commits a medida que avanzan**, no uno solo gigante al final: sirve para ustedes, para volver atrás cuando algo deja de andar, y sirve para la corrección.

En el repositorio tienen que estar el código, los reportes de caos y el informe en PDF. Los commits fuera de la fecha límite hacen que el trabajo se considere **tarde**.

4.3 Fecha de entrega

La fecha límite es el **viernes 25 de septiembre a las 23:59**. Como siempre, es preferible entregar tarde que no entregar, pero cuanto más tarde se entregue, menos probable es que se corrija en tiempo y forma.